Chapter 10
Deployment Strategies and Automation

10.1 Deployment Strategies and Automation (CI/CD)

Deployment means placing application code on a server so that end users can access and use it.

Example:
1. A developer fixes a bug in a website
2. The updated code is uploaded to the server
3. Users now see the corrected version of the website

Deployment is a critical step because it directly affects users and business continuity.

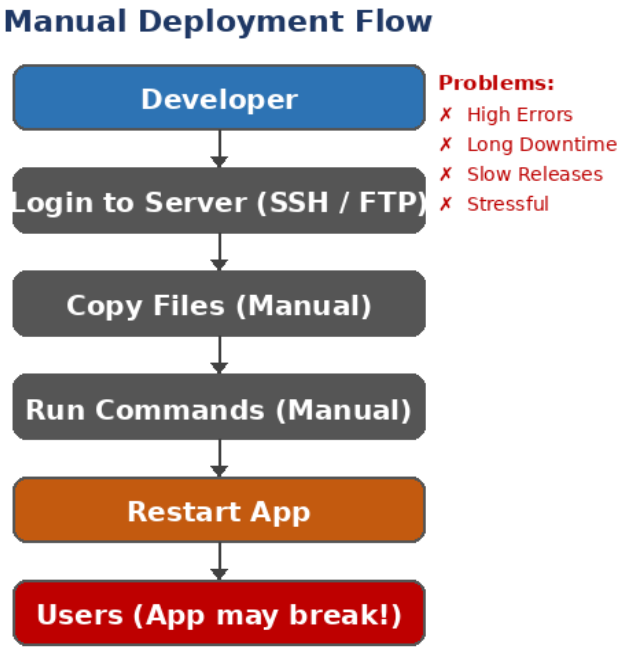
10.1.1 The Pain of Manual Deployment

(The "Stone Age" of Software Deployment)

Before automation, deployment was done manually by humans — slow, risky, and error-prone.

Steps of Manual Deployment:
4. Stop the running application / Website goes offline (downtime begins)
5. Login to server (SSH / FTP)
6. Copy files manually
7. Run commands manually:
    - Install dependencies
    - Run database updates
8. Restart the application / Hope everything works

Manual Deployment Flow Diagram:



Problems with Manual Deployment:

- High Errors – Humans may miss steps or type wrong commands
- Long Downtime – Application remains offline during deployment
- Slow Releases – Teams fear deployment and delay updates
- Stressful – Often done late nights or weekends

Solution: Deployment Automation

To solve these problems, deployment steps are automated:

9. Fast
10. Repeatable
11. Reliable
12. Error-free

This automation leads to CI/CD (Continuous Integration & Continuous Delivery/Deployment).

DEPLOYMENT AND DevOps

DEPLOYMENT STRATEGIES

Deployment strategies are methods used to release new versions of applications (frontend, backend, full stack systems) into production while ensuring minimal downtime, high reliability, and safe rollbacks in case of failure.

They build on DevOps deployment principles but focus on how entire full stack applications (frontend + backend + database) are rolled out. Each strategy balances speed, stability, and user impact differently. In simple words - it is how updates are released to users.

Common Deployment Strategies

The various common deployment strategies are discussed as:

1. Recreate Deployment (Traditional Deployment)—In the recreate strategy, the old version of an application is completely shut down before the new version is deployed.

Process:
(i) Stop the running application (frontend, backend, or API).
(ii) Deploy the new version.
(iii) Start the application again with the updated code.

Use Case:
- Suitable for small internal apps or non-critical systems where downtime is acceptance.

2. Rolling Deployment—The rolling strategy updates application instances in batches, one after another, until all old versions are replaced.

Process:
(i) Divide all running servers or containers into batches.
(ii) Update one batch with the new version while others continue serving traffic.
(iii) Gradually replace all instances.

Use Case:
- It is ideal for web applications, microservices, or cloud-based deployments.

3. Blue-Green Deployment—Two identical environments are maintained -
- Blue: Current production environment.
- Green: New version environment.
Traffic is switched from Blue → Green once the new version is tested and ready.

Process:
(i) Deploy the new version to the Green environment.
(ii) Test and verify it.
(iii) Redirect production traffic from Blue → Green.
(iv) Keep Blue for rollback if needed.

Use Case:
- It is used in enterprise systems, e-commerce, and financial apps where uptime is critical.

4. Canary Deployment—The new version is first released to a small subset of users (like 5-10%), then gradually rolled out to everyone.

Process:
(i) Deploy new version to a small group of servers/users.
(ii) Monitor for errors, performance, and user experience.
(iii) Gradually increase rollout if stable.

Use Case:
- Large-scale applications (Google, Facebook, Netflix) use this to safely test updates.

5. Shadow Deployment—The new version runs in parallel with the current version and receives a copy of real traffic, but users only see the old version's response.

Process:
(i) Mirror live user traffic to the new version.
(ii) Compare results, performance, and logs without affecting real users.

Use Case:
- It is used for backend testing, database migration, or machine learning models before production.

6. A/B Testing Deployment—Two different versions (A and B) of the app run simultaneously. A subset of users is sent to each version to compare which performs better.

Process:
(i) Deploy two versions of the app (A and B).
(ii) Route a percentage of users to each.
(iii) Measure engagement, performance, and satisfaction.
(iv) Choose the better-performing version.

Use Case:
- Ideal for UI/UX testing, feature experimentation, or marketing variations.

8.2 DevOps

DevOps is a combination of "Development" and "Operations."

It is a set of practices, tools, and cultural philosophies that bring together software development (Dev) and IT operations (Ops) to shorten the development life cycle, improve collaboration, and deliver high-quality software faster and more reliably.
DevOps is a way to automate and integrate the processes between developers and IT teams so they can build, test, and release software quickly and efficiently.

8.2.1 Key Goals of DevOps
(i) Faster Delivery: Deploy new features, fixes, and updates rapidly.
(ii) Automation: Automate repetitive tasks such as testing, deployment, and monitoring.
(iii) Collaboration: Break down silos between development and operations teams.
(iv) Reliability: Improve software stability and performance.
(v) Continuous Improvement: Enable ongoing feedback and iteration.

8.2.2 Core Components of DevOps
The core components of DevOps are given below:

S.No.	Component	Description
(i)	Continuous Integration (CI)	Developers frequently merge code into a shared repository, and automated builds/tests verify it.
(ii)	Continuous Delivery (CD)	Code is automatically prepared for release to production after CI verification.
(iii)	Infrastructure as Code (IaC)	Infrastructure is managed and provisioned through code (e.g., Terraform, Ansible).
(iv)	Monitoring and Logging	Continuous tracking of application and infrastructure health.
(v)	Collaboration and Communication	Use of shared tools and agile culture for transparency.

8.2.3 Key DevOps Tools
The various DevOps tools are given in table below:

S.No.	Stage	Purpose	Tools
(i)	Source Code Management	Version control	Git, GitHub, GitLab, Bitbucket
(ii)	Continuous Integration	Build and test automation	Jenkins, GitHub Actions, Travis CI, CircleCI
(iii)	Configuration Management	Manage infrastructure	Ansible, Puppet, Chef
(iv)	Containerization	Package apps with dependencies	Docker, Podman
(v)	Container Orchestration	Manage multiple containers	Kubernetes, OpenShift
(vi)	Monitoring	Track app performance	Prometheus, Grafana, ELK Stack
(vii)	Cloud Platforms	Host and scale applications	AWS, Azure, GCP

8.2.4 DevOps Life Cycle (Phases)
The various DevOps Life Cycle phases are given as:
(i) Plan: Define requirements and project scope using tools like Jira or Trello.
(ii) Code: Developers write and manage code using Git repositories.
(iii) Build: Code is compiled and built automatically.
(iv) Test: Automated tests ensure code quality and detect bugs early.
(v) Release: Approved builds are deployed to staging or production environments.
(vi) Deploy: Continuous deployment releases software updates automatically.
(vii) Operate: Monitor performance, availability, and reliability of apps.
(viii) Monitor: Collect feedback to improve future releases.

These phases form a continuous loop, ensuring ongoing improvement.

Fig. 8.1

8.2.5 Advantages of DevOps
DevOps brings together development and operations teams to deliver software more efficiently, with better quality and reliability.
Below are the key advantages of adopting DevOps in an organization.

1. Faster Software Delivery—
- DevOps practices like Continuous Integration (CI) and Continuous Delivery (CD) automate build, testing, and deployment.

- This leads to shorter development cycles, allowing faster release of new features or updates.
- Example: A bug fix that earlier took weeks can now be deployed within hours.

2. Improved Collaboration and Communication—
- DevOps removes the traditional barrier between developers (who write code) and operations (who deploy and manage it).
- Both teams work together throughout the lifecycle — from development to deployment and maintenance.
- This fosters a culture of shared responsibility and better coordination.

3. Increased Deployment Frequency—
- With automation and CI/CD pipelines, updates can be pushed frequently and safely.
- Organizations can release multiple updates per day instead of quarterly or monthly cycles.
- Outcome: Faster innovation and quicker response to market needs.

4. Better Resource Utilization—
- DevOps encourages use of technologies like containerization (Docker) and cloud computing, which make efficient use of system resources.
- Automation reduces the need for manual intervention, freeing developers for higher-value tasks.

5. Enhanced Automation and Efficiency—
- Tasks such as builds, tests, deployments, and monitoring are automated.
- This reduces manual errors and saves time.
- Tools like Jenkins, Ansible, and Terraform streamline processes end to end.

6. Improved Security (DevSecOps)—
- Security practices are integrated into every stage of the development pipeline.
- Automated security scans and configuration management ensure compliance and reduce vulnerabilities.
- Known as DevSecOps, this approach builds security into the development process instead of adding it later.

7. Continuous Monitoring and Feedback—
- DevOps uses monitoring tools like Prometheus, Grafana, and ELK Stack to track performance and errors in real time.
- Continuous feedback helps teams make quick improvements and reduce downtime.

8. Scalability and Flexibility—
- Cloud-based and containerized environments make it easy to scale applications up or down based on traffic.
- DevOps supports microservices architecture, allowing independent scaling of components.

10.1.2 Continuous Integration (CI)

(The Daily Health Check)

Definition:

Continuous Integration (CI) is a practice where every code change is automatically built and tested whenever a developer pushes the code to a shared repository.

Main Goal of CI:

- Detect bugs early
- Keep the main codebase stable and working

10.1.2.1 CI Process

13. Code Commit & Push – Developer writes code and pushes it to GitHub
14. Automatic Trigger – The repository automatically notifies the CI tool

Common CI Tools:

- GitHub Actions
- Jenkins
- GitLab CI

Build Stage :

- Install Dependencies
- Compile/Build the application
- Example : npm install
  - npm run build

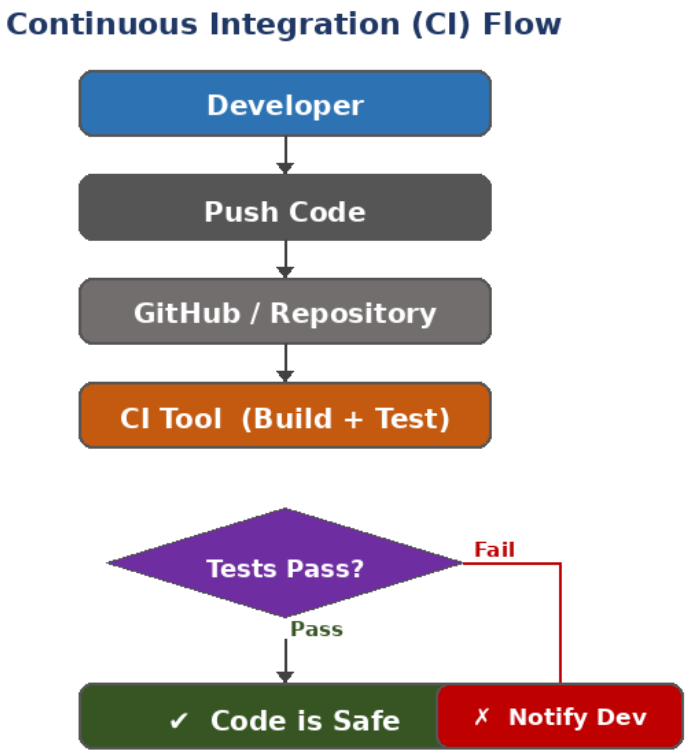
Test Execution:

- Unit Tests – Test small functions
- Integration Tests – Test component interaction

Feedback to Developer:

- Tests pass → Code is safe
- Tests fail → Developer is notified immediately

CI Flow Diagram:



10.1.3 Continuous Delivery vs Continuous Deployment

After CI , the next step is releasing the application. This is done using Continuous Delivery or Continuous Deployment.

(a) Continuous Delivery

"Everything is ready. Deploy when you approve."

- Manager reviews application
- Clicks Deploy to Production

(b) Continuous Deployment

"If tests pass, deploy immediately."

- Code passes all tests
- Goes live automatically at 11:30 AM

Comparison:

Feature	Continuous Delivery (CDel)	Continuous Deployment (CDep)
Meaning	Code is ready to deploy	Code is deployed automatically
Human Approval	Required	Not required

Automation Level	Stops before production	Goes fully to production
Risk Level	Lower	Higher
Used In	Banking, Healthcare	Startups, Social Media

Summary

Deployment automation using CI/CD transforms software delivery from a slow, risky process into a **fast, reliable, and repeatable workflow.**

- CI ensures code quality through automated building and testing
- Continuous Delivery ensures release readiness
- Continuous Deployment enables rapid innovation

Together, CI/CD allows organizations to deliver software **frequently, safely, and confidently.**

10.2 CI/CD Pipelines

A CI/CD pipeline is an automated step-by-step process that takes your code from a developer's computer and safely delivers it to users.

Think of it like an **assembly line in a factory:**

1. Each stage checks quality
2. If something is wrong, the process stops
3. Only good code reaches to users

Why Do We Need a CI/CD Pipeline?

Without a pipeline:

15. Deployment is manual
16. Errors are common
17. Releases are slow and risky

With a pipeline:

18. Everything is automated
19. Bugs are caught early
20. Software is delivered faster and safer

10.2.1 Main Stages of a CI/CD Pipeline

10.2.1 Main Stages of a CI/CD Pipeline

1. **Source Stage (Trigger)**
   In this stage, a developer writes code and pushes it to GitHub / GitLab.
   This action starts the pipeline automatically.

   Example:
   `git push origin main`

   Output:
   Source code

2. **Build Stage (Packaging)**
   In this stage:

   1. Dependencies are installed
   2. Code is compiled (if required)
   3. A runnable package is created
3. **Example:**
   Node.js: npm install
   Java: mvn package
   Docker: build image

Output:
Artifact (ready-to-run application)

4. **Test Stage (Validation)**
   Automated tests are executed.

   (a) **Unit Tests**
   Test small pieces of code

   Example: “Does this function return correct results?”

   (b) **Integration Tests**
   Test how components work together

   Example: “Can the app connect to database?”

   (c) **Security & Performance Tests (Optional)**

   1. Check vulnerabilities
   2. Check load handling
5. **Rule:**
   If any test fails, the pipeline stops

   Output:
   Verified and safe code

6. **Deploy Stage (Staging Environment)**
   Application is deployed to **Staging**

   What is Staging?

   1. A copy of production
   2. Used for final testing
   3. No real users affected

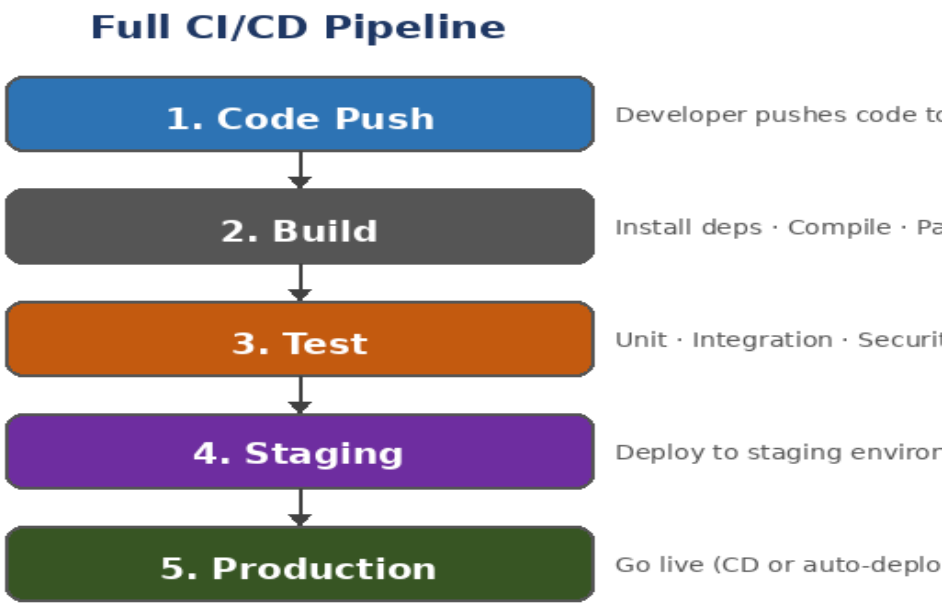
   Purpose:
   Final verification before going live

7. **Production Stage (Release)**
   Two possibilities:

   1. **Continuous Delivery (CDE)**
      (i) Human approval required
      (ii) Manager clicks “Deploy”
   2. **Continuous Deployment (CDep)**
      (i) Fully automatic
      (ii) If tests pass, then goes live

   Goal:
   Minimal or zero downtime

Full CI/CD Pipeline Diagram:



Legend:

- CDel = Human approves deploy
- CDep = Auto deploy on green tests

Example: Steps for updating a Website Button Color

1. Developer changes button color
2. Pushes code to GitHub
3. Pipeline starts automatically
4. Code is built
5. Tests are run
6. Deployed to staging
7. Approved (or auto-deployed)

8. **Users see new button color**

10.2.2 Tools Used in CI/CD Pipelines

Purpose	Tools
Source Control	Git, GitHub, GitLab
CI/CD	GitHub Actions, Jenkins, GitLab CI
Build	Maven, Gradle, npm, yarn
Containerization	Docker
Cloud	AWS, Azure, GCP

Source Control – A system that tracks and manages changes to code over time.

Git – A distributed version control system used to track code changes locally and remotely.

GitHub – A cloud-based platform for hosting Git repositories and collaborating on code.

GitLab – A DevOps platform that provides Git repository management along with CI/CD features.

CI/CD – Continuous Integration and Continuous Deployment/Delivery, a practice to automate building, testing, and releasing code.

GitHub Actions – A CI/CD tool within GitHub that automates workflows like testing and deployment.

Jenkins – An open-source automation server used to build, test, and deploy applications.

GitLab CI – Built-in CI/CD tool in GitLab for automating pipelines.

Build – The process of converting source code into executable applications.

Maven – A build automation tool mainly used for Java projects.

Gradle – A flexible build automation tool that supports multiple languages like Java and Kotlin.

npm – Node Package Manager, used to manage JavaScript packages and dependencies.

yarn – A fast and reliable JavaScript package manager similar to npm.

Containerization – Packaging an application and its dependencies into a portable container.

Docker – A platform used to create, run, and manage containers.

Cloud – Remote servers and services used to store, manage, and process data over the internet.

AWS – Amazon Web Services, a cloud platform offering computing, storage, and other services.

Azure – Microsoft's cloud computing platform providing various cloud services.

GCP – Google Cloud Platform, a suite of cloud services offered by Google.

10.2.3 Advantages of CI/CD Pipelines

21. Faster Releases – Automation reduces time
22. Fewer Bugs – Early testing
23. Less Downtime – Safe deployments
24. Better Teamwork – Easy integration
25. Reliable – Same process every time

10.3 Containerization with Docker

The "It Works on My Machine" Problem

A developer says it works on their laptop, but when deployed to a server: app crashes, dependencies missing, or different OS/library versions.

The Root Cause

- Operating System (OS)
- Runtime (Node, Java, Python, etc.)
- Libraries and dependencies
- Environment variables and configurations

This problem is called:

- "It Works on My Machine"
- Dependency Hell

Developer Laptop	Production Server
Node 18	Node 16
Library v2.1	Library v2.0
Linux A	Linux B
App Works ✔	App Fails ✗

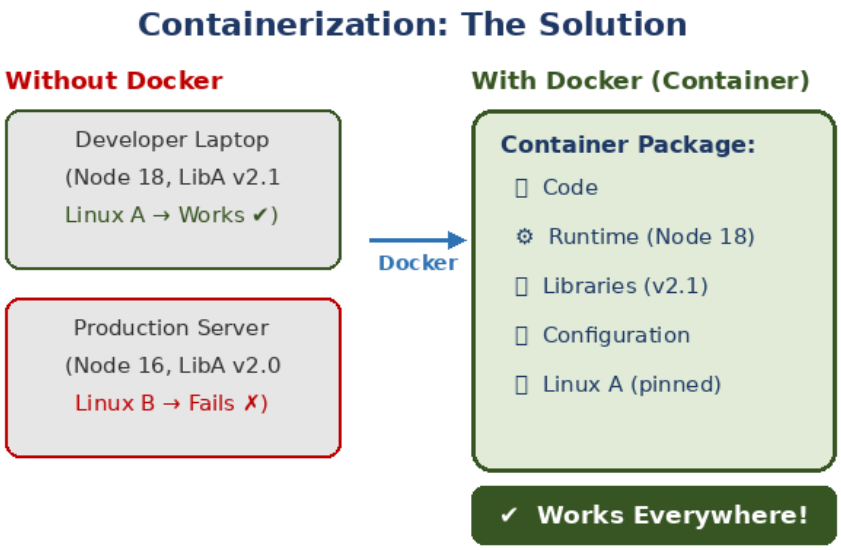
The Solution: Containerization

Containerization bundles the app and its environment into one package. If it works once, it works everywhere.

Definition 1:
Containerization means packaging an application together with everything it needs to run, so it works the same everywhere.

Definition 2:
A container is a package that includes the application and everything required to run it.

Containerization Diagram:



10.3.2 The Docker Lifecycle

Docker operates through a three-step chain: Dockerfile → Docker Image → Docker Container

26. Dockerfile (The Instructions) – A text file telling Docker:
    - Which OS/runtime to use
    - Which files to copy
    - Which commands to run

> **Analogy:**
> Dockerfile = Recipe

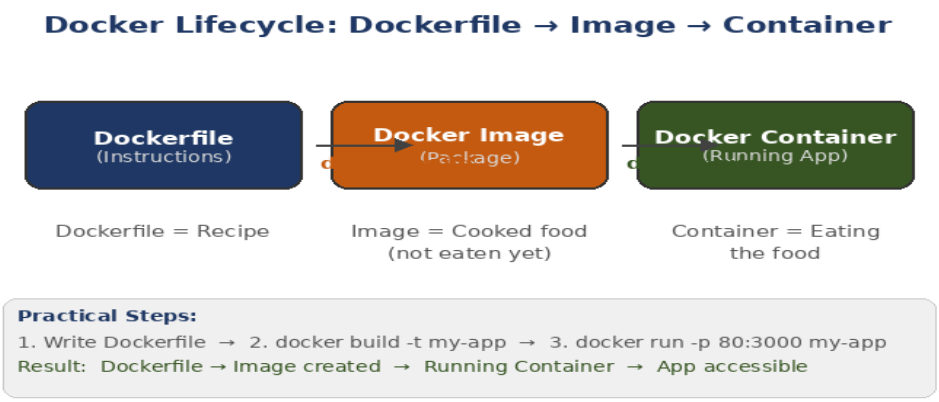
27. Docker Image (The Package) – Built from a Dockerfile:
    - A ready-to-use application package
    - Not running yet

> **Analogy:**
> Image = Cooked food (not eaten yet)

28. Docker Container (The Running App):
    - A running instance of an image
    - The live application

> **Analogy:**
> Container = Eating the food

Docker Lifecycle Diagram:



10.3.2.1 Practical Implementation

Dockerfile Example:

```
FROM node:18-alpine
WORKDIR /app
COPY . .
RUN npm install
CMD ["npm", "start"]
```

This says:

- Use Node 18
- Copy app
- Install dependencies
- Start app

Build Image:

```
docker build -t my-app
```

Result: Dockerfile → Image created

Run Container:

```
docker run -p 80:3000 my-app
```

Result: Image → Running Container → App is now running inside a container.

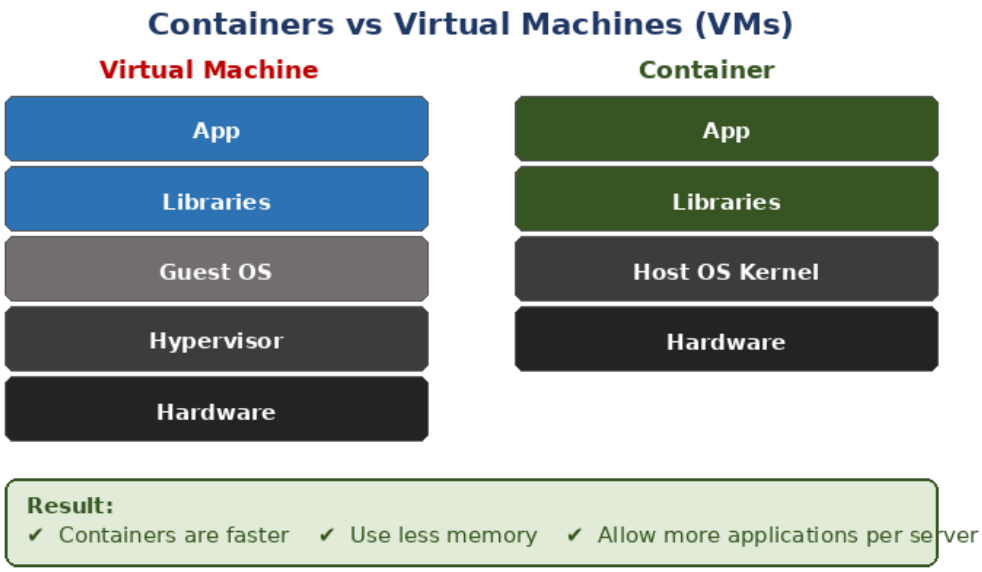
10.3.4 Containers vs Virtual Machines (VMs)

Both provide isolation, but Docker containers are significantly more efficient.

Virtual Machines:
Include a full Guest OS — heavy, slow to start, resource-intensive. Includes: App, Libraries, Guest OS, Hardware.

Containers:
Share the Host OS Kernel — lightweight, start in seconds, use much less memory. Includes: App, Libraries, Host OS Kernel, Hardware.

Containers vs VMs Diagram:



Result:
- Containers are faster
- Use less memory
- Allow more applications per server

10.3.5 Advantages of Docker & Containerization

29. Guaranteed Consistency – Same environment everywhere
30. Fast Deployment – Containers start in seconds
31. Resource Efficiency – Run many containers on one machine
32. Simplified CI/CD – CI builds Docker image; CD deploys same image; no environment mismatch

Summary:
33. Problem: "It works on my machine"
34. Solution: Containerization
35. Tool: Docker
36. Core Flow: Dockerfile → Image → Container
37. Result: Same application runs everywhere
38. Impact: Fast deployment and reliable CI/CD

10.4 Deployment Platforms

After developing an application, the next challenge is deploying it so users can access it. Modern DevOps practices use CI/CD pipelines to automate building and testing, and Docker containers to package applications. These containers need a cloud platform where they can run.

> **Definition:**
> Deployment platforms are cloud services that host and run applications and services environments, broadly classified into Platform as a Service (PaaS) and Infrastructure as a Service (IaaS).

Complete Application Delivery Flow:
Source Code → CI/CD Pipeline → Docker Image → Deployment Platform → Users

10.4.1 "As a Service" Models: PaaS vs IaaS

(A) Infrastructure as a Service (IaaS)

The cloud provider gives basic infrastructure; the user manages everything else.
User manages:

39. Operating System (OS)
40. Runtime and middleware
41. Security patches
42. Application code and data

Provider manages:

43. Physical servers
44. Virtualisation

(B) Platform as a Service (PaaS)

The cloud provider manages most infrastructure details.
User manages:

45. Application code
46. Application data

Provider manages:

47. OS
48. Runtime
49. Middleware
50. Servers, Storage, and Networking

Feature	IaaS	PaaS
Control	Very high	Limited
Complexity	High	Low
Ease for beginners	Difficult	Easy
Focus	Infrastructure	Application

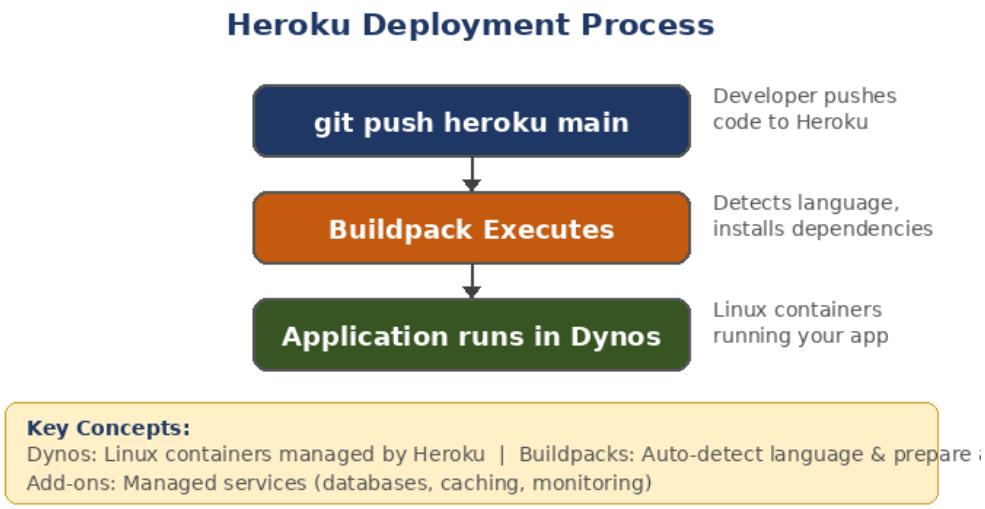
10.4.2 Heroku – A Beginner-Friendly PaaS

Heroku is a Platform as a Service (PaaS) designed to simplify deployment of web apps and backend applications.

Key Concepts:

51. Dynos – Heroku runs applications inside managed Linux containers called Dynos
52. Buildpacks – Automatically detect programming language and prepare the app for execution
53. Add-ons – Managed services such as databases, caching, and monitoring

Heroku Deployment Diagram:



Use Cases:

54. Rapid prototyping
55. Student projects
56. Backend APIs and full-stack applications

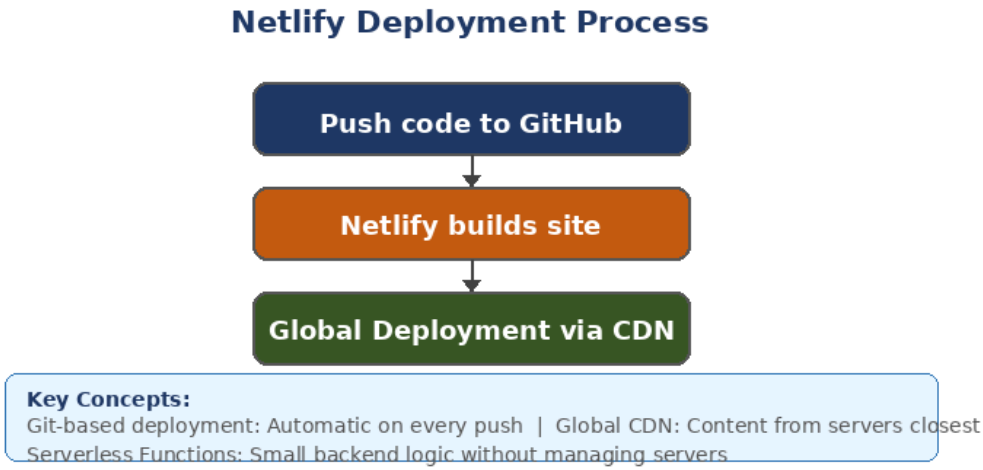
10.4.3 Netlify – Frontend Deployment Platform

Netlify is a modern PaaS platform optimized for frontend and static websites.

Key Concepts:

57. Git-based Deployment – Automatic deployment on every code push
58. Global CDN – Content is served from servers closest to users
59. Serverless Functions – Small backend logic without managing servers

Netlify Deployment Diagram:



Use Cases:

60. Static websites
61. Portfolios and blogs
62. React, Angular, and Vue applications

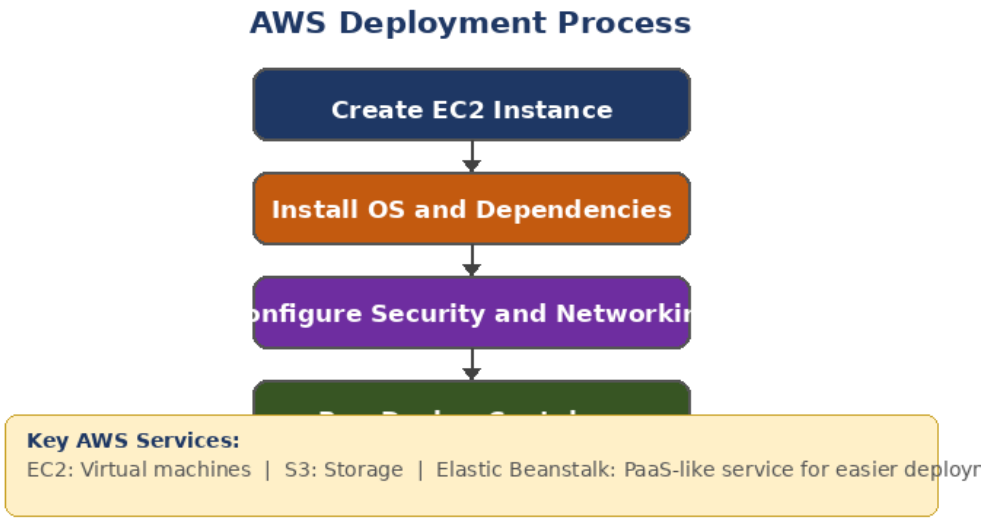
10.4.4 AWS – Infrastructure as a Service (IaaS)

Amazon Web Services (AWS) is primarily an IaaS provider offering maximum flexibility and control.

Key Services:

63. EC2 – Virtual machines where users install OS, Docker, and applications
64. S3 – Storage service for files and static websites
65. Elastic Beanstalk – AWS's PaaS-like service for easier deployments

AWS Deployment Diagram:



Use Cases:

66. Large-scale, custom cloud infrastructure
67. Advanced DevOps environments

Comparative Summary

Platform	Service Model	Ease of Use	Scale of Use
Heroku	PaaS	Full-stack apps	Easy
Netlify	PaaS	Frontend/static	Very Easy
AWS	IaaS	Full control & scalability	Difficult

10.5 DevOps and Infrastructure Management

Modern software development uses powerful tools such as CI/CD pipelines, Docker containers, and cloud-based deployment platforms. However, tools alone are not sufficient. Organizations need a cultural and operational approach called DevOps.

> **Definition:**
> DevOps is a combination of Development (Dev) and Operations (Ops). It is a cultural movement and set of practices that aims to shorten the software development life cycle and deliver high quality software continuously.

(A) Traditional Problem (Before DevOps)

Earlier, development and operations teams worked in separate silos:
Developers (Dev):

- Focused on writing new features quickly

Operations (Ops):

- Focused on system stability and uptime

This created conflicts:

- Developers wanted fast releases
- Operations wanted fewer changes

Result:

68. Slow deployments
69. High-risk releases
70. "Throw it over the wall" mentality (Dev hands code to Ops)

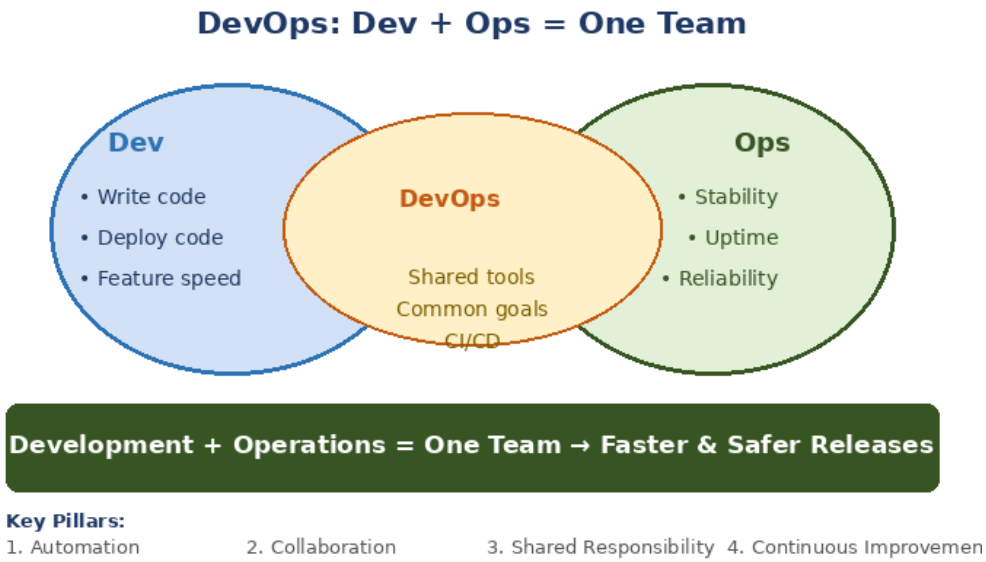
(B) DevOps Goal

DevOps promotes shared responsibility and collaboration:
Development + Operations = One Team → Faster & Safer Releases

The same team is responsible for:

71. Writing code
72. Deploying code
73. Running code in production

DevOps Culture Diagram:



Key Pillars of DevOps Culture

74. Automation
    - Automates repetitive tasks: Testing, Deployment, Infrastructure provisioning
    - Reduces human error
    - Increases speed

75. Collaboration
    - Shared tools, shared metrics, common goals
    - Encourages teamwork instead of blame

76. Shared Responsibility
    - Developers responsible for how apps behave in production
    - Operations teams help developers deploy safely
    - Both teams own system reliability

77. Continuous Improvement
    - Constant monitoring and feedback
    - Verify bottlenecks
    - Improve performance and deployment processes

10.6 Infrastructure as Code (IaC)

(A) Traditional Infrastructure Management Problem

Earlier, servers were configured manually:

78. Logging into servers
79. Installing software by hand
80. Clicking through settings

This caused:

81. Inconsistency between servers
82. Configuration drift (servers become different over time)
83. Difficult recovery

> **Definition:**
> Infrastructure as Code (IaC) is the practice of managing and provisioning infrastructure resources using code files, instead of manual processes.

Infrastructure includes:

84. Virtual machines
85. Networks
86. Databases
87. Load balancers

10.6.1 Why IaC is Important

88. Consistency
    - Infrastructure is defined in code (blueprints)
    - Running the same code creates identical services

89. Reproducibility
    - Entire environments can be destroyed and rebuilt easily
    - Same setup for development, testing, and production

90. Version Control
    - Infrastructure code is stored in Git
    - Changes are tracked
    - Easy rollback to previous versions

10.6.2 Categories of IaC Tools

IaC tools are mainly divided into:

91. Provisioning tools
92. Configuration management tools

10.6.2.1 Terraform – Provisioning Tool

Terraform is an open-source provisioning tool developed by HashiCorp.

(a) Purpose:
Creates and manages cloud resources such as:
- Virtual machines
- Networks
- Storage
- Security rules

(b) Key Concept: Desired State:
Terraform uses a declarative approach:
- You define what infrastructure you want
- Terraform decides how to create or modify it

Example (Conceptual):

```
resource "aws_instance" "app_server" {
  ami           = "ami-0123456789"
  instance_type = "t2.micro"
  tags = {
    Name = "MyWebAppServer"
  }
}
```

This code declares:
- A virtual machine
- Its size
- Its name

(c) Key Advantages of Terraform:
93. Cloud agnostic
94. Same tool works with AWS, Azure, and Google Cloud

10.6.2.2 Ansible – Configuration Management Tool

Ansible is a configuration management tool used to manage software inside servers.

(a) Purpose – Installs and configures:
- Software
- Packages
- Users
- Configuration files

Runs on:
- Virtual machines
- Containers

(b) Key Concepts:
95. Agentless:
    - No agent required on servers
    - Uses SSH (Linux) or WinRM (Windows)

96. Playbooks:
    - Configuration written in YAML
    - Easy to read and understand

Example (Conceptual):

```
- name: Install Nginx web server
  ansible.builtin.apt:
    name: nginx
    state: latest
```

This ensures:

- Nginx is installed
- Always at latest version

(c) Key Advantages of Ansible:

97. Easy for beginners
98. Simple YAML syntax
99. Human-readable configuration

Terraform vs Ansible Comparison:

Terraform vs Ansible: IaC Tools

	Terraform	Ansible
Tool Type	Provisioning	Configuration Management
Focus	Creates infrastructure	Configures software inside it
Language	HCL (HashiCorp Config Lang)	YAML Playbooks
Agentless	No	Yes – uses SSH/WinRM
Use For	AWS, Azure, GCP resources	Install apps, packages, users

Tool	Focus	Role
Terraform	Provisioning	Creates infrastructure
Ansible	Configuration	Configures software inside infrastructure

10.7 Monitoring and Logging

Deploying an application does not end with pushing code to a server. The most critical requirement is ensuring that the application remains healthy, fast, and available 24/7.

This responsibility is handled through Monitoring and Logging, which allow teams to observe system behavior, detect failures early, and troubleshoot issues efficiently.

1. Logging (The Black Box Recorder)

Logging is like the black box of an airplane. When a failure occurs, logs show the exact sequence of events that led to the issue.

Logs are:

- Generated by application code
- Text-based messages
- Used as a record for root cause analysis

Example log message:

```
ERROR: User authentication failed – Invalid API Key
```

2. Monitoring (The Live Dashboard)

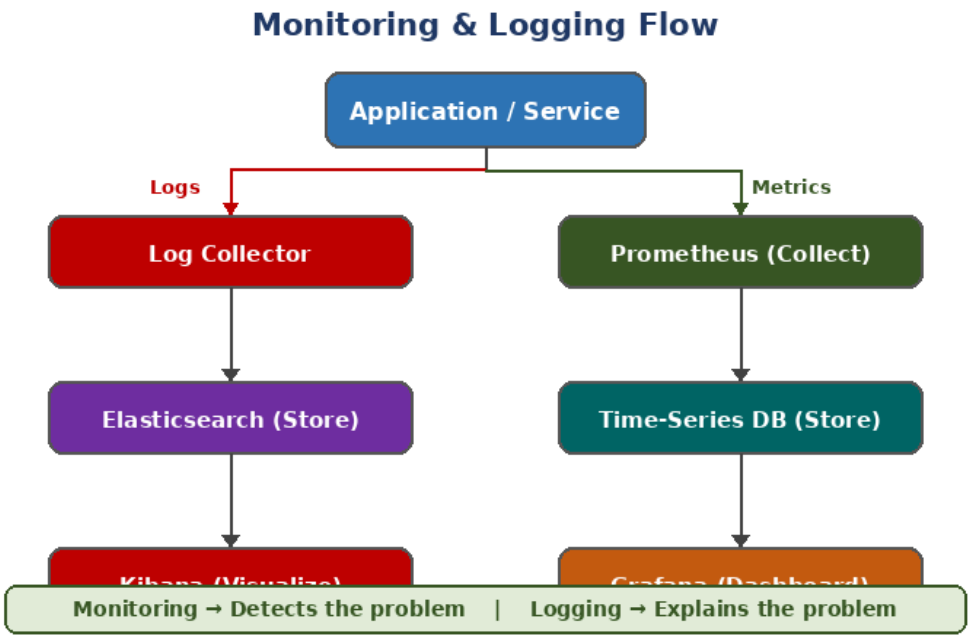
Monitoring is like the dashboard of a car. It shows real-time system health.

- Tracks performance continuously
- Helps detect problems before users are affected

Example metrics:

- CPU Usage: 80%
- Response time: 200 ms
- Error rate: 1%

Monitoring & Logging Flow Diagram:



10.7.1 Prometheus

Prometheus is an open-source monitoring system designed to collect and store time-series metrics (meaning metrics that change over time).

(a) How Prometheus Works (Pull Model)

Unlike traditional systems that receive data automatically (push model), Prometheus uses a pull-based approach.

Step-by-Step Working:

100. Targets / Exporters:
    - Applications expose a metrics HTTP endpoint
    - Exporters convert system data into Prometheus format

101. Scraping:
    - Prometheus periodically pulls data (every 15–30 seconds)

102. Storage:
    - Metrics are stored in a time series database

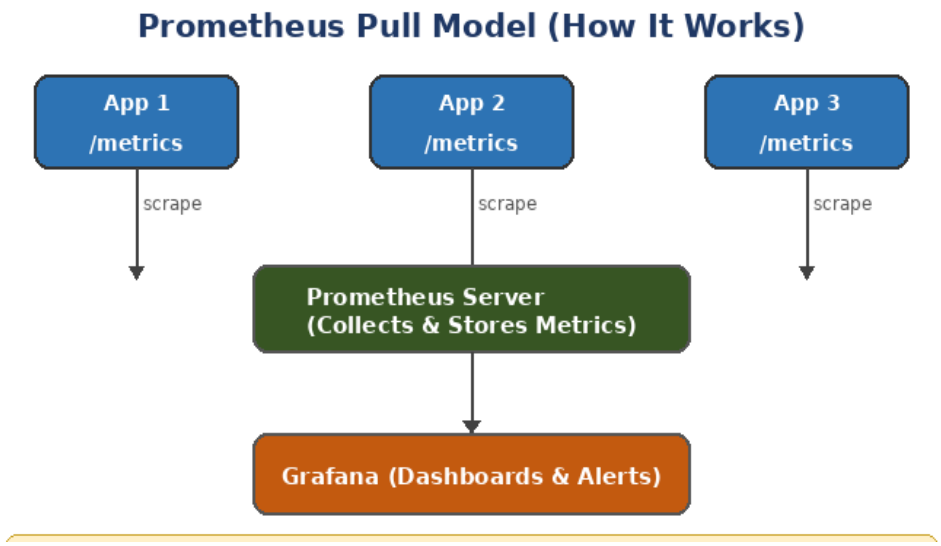
(b) PromQL (Prometheus Query Language):
Prometheus provides PromQL, a powerful query language to analyse metrics.
Example:

```
rate(http_requests_total[5m])
```

Meaning:

- Calculates the average request rate over the last 5 minutes
- Used to measure traffic load

Prometheus Pull Model Diagram:



(c) Role of Prometheus in DevOps

103. Tracks system performance
104. Detects abnormal behaviour
105. Provides data for alerting and dashboards

10.7.2 Grafana (The Visualization Engine)

Raw metrics stored in Prometheus are hard to interpret. Grafana converts those metrics into visual dashboards.

(a) Why Grafana is Needed:
Raw metrics stored in Prometheus are hard to interpret. Grafana converts those metrics into visual dashboards.

(b) Key Features of Grafana:

106. Connects to Prometheus as a data source
107. Displays graphs, charts, and gauges
108. Provides real-time insights into application health

Example (Dashboard Questions Answered):

- How many errors are users seeing?
- Is memory usage approaching limits?
- Are response times increasing?

(c) Alerting with Grafana and Prometheus:
Alerts ensure teams receive proactive issue notification.

Example alert rule:

If error rate > 5% for more than 5 minutes, notify the Ops team via Slack.
This prevents small problems from becoming major outages.

10.7.3 Centralized Logging

(a) Problem with Traditional Logging

In modern systems:

109. Applications run on multiple servers and containers
110. Logs are scattered across machines
111. Manual log inspection is impossible

Solution – Centralized Logging:
All logs are collected in one central location, making them:

- Searchable
- Filterable
- Easy to analyse

(b) ELK Stack (Elastic Stack)

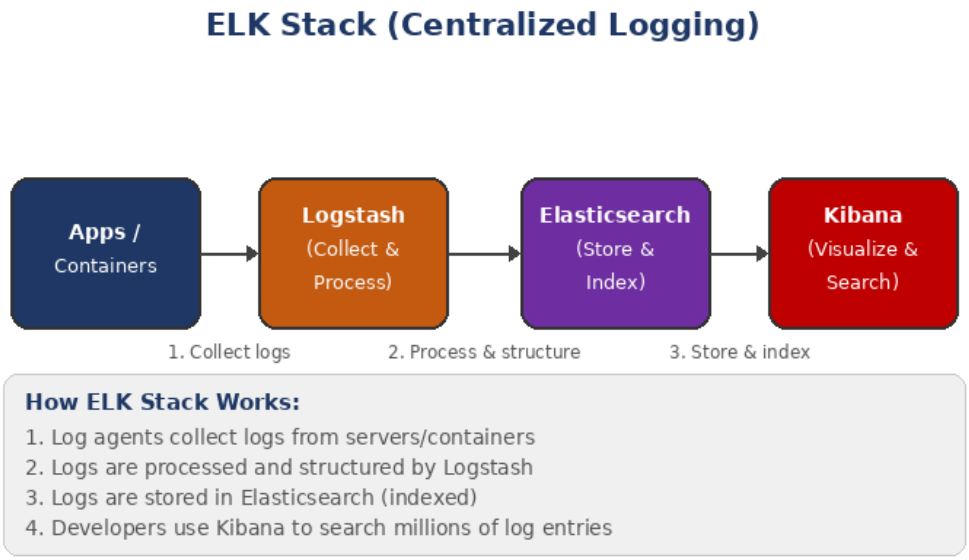
The most popular centralized logging solution is the ELK Stack.

Component	Role
Elasticsearch	Stores and indexes logs
Logstash	Collects and processes logs
Kibana	Visualises and searches logs

How ELK Stack Works:

112. Log agents collect logs from servers/containers
113. Logs are processed and structured by Logstash
114. Logs are stored in Elasticsearch (indexed)
115. Developers use Kibana to search millions of log entries

ELK Stack Diagram:



(c) Relationship Between Monitoring and Logging

Monitoring → Detects the problem
Logging → Explains the problem

Both are essential for:

116. High availability
117. Fast recovery
118. Reliable production systems

Chapter 10 – Complete Summary

Chapter Summary Table:

Chapter 10 - Complete Summary

Step	Focus Area	Tools	Impact
1	CI/CD	GitHub Actions	Faster, tested releases
2	Containerization	Docker	Environment consistency
3	Deployment	AWS, Netlify	User accessibility
4	Infrastructure	Terraform, Ansible	Scalable environments
5	Operations	Prometheus, Grafana	Reliability and monitoring

Step	Focus Area	Tools	Impact
1	CI/CD	GitHub Actions	Faster, tested releases
2	Containerization	Docker	Environment consistency
3	Deployment	AWS, Netlify	User accessibility
4	Infrastructure	Terraform, Ansible	Scalable environments
5	Operations	Prometheus, Grafana	Reliability and monitoring

Exercise Questions

119. What is Deployment?
120. What are the problems with Manual Deployment?
121. What is Continuous Integration (CI)?
122. Explain various steps of CI Process.
123. Distinguish between Continuous Delivery and Continuous Deployment.
124. What is CI/CD Pipelines?
125. Explain various stages of CI/CD Pipelines.
126. Explain various tools Used in CI/CD Pipelines.
127. Explain various advantages of CI/CD Pipelines.
128. What is Containerization?
129. What is Docker? What are its responsibilities?
130. Explain the Docker Lifecycle (Dockerfile → Image → Container).
131. What is the difference between Containers and Virtual Machines?
132. What are Deployment Platforms? Explain PaaS vs IaaS.

133. Explain Heroku, Netlify, and AWS as deployment platforms.
134. What is DevOps? Explain the Key Pillars of DevOps Culture.
135. What is Infrastructure as Code (IaC)?
136. Explain Terraform and Ansible with examples.
137. What is Monitoring and Logging? Why are they important?
138. Explain Prometheus and Grafana.
139. What is the ELK Stack? How does it work?

Program : 15

Deployment Pojects : Set Up a CI/CD pipeline and deploy a web application to a cloud platform (like Netlify)

We will:

- Create a simple HTML website
- Upload it to GitHub
- Deploy automatically using Netlify

📁 Step 1: Create Website Code

Create a file named `index.html`:

```
<!DOCTYPE html>
<html>
<head>
  <title>My CI/CD App</title>
</head>
<body>
  <h1>Hello World 🎉</h1>
  <p>My CI/CD pipeline using Netlify is working!</p>
</body>
</html>
```

📤 Step 2: Upload to GitHub

1. Open GitHub
2. Create a new repository

3. Upload your `index.html` file
4. Click **Commit**

⚙️ Step 3: Deploy Using Netlify

1. Go to Netlify website
2. Sign up / Login using GitHub
3. Click **“Add new site”** → **“Import from Git”**
4. Choose your GitHub repository
5. Click **Deploy site**

🎉 Done! Your website is live

🔁 CI/CD Working

You change `index.html`

- Push changes to GitHub
- Netlify **automatically redeploys** your website

👉 This is CI/CD:

- **CI** = code update
- **CD** = automatic deployment

✅ Output

Your live website will show:

Hello World 🎉
My CI/CD pipeline using Netlify is working!

🔁 How to Push Changes to GitHub

After you edit your `index.html`, you need to **upload (push) the updated file**.

✅ Method 1: Easiest (No Software Needed)

Steps:

1. Go to your repository on GitHub
2. Click on `index.html`
3. Click ✏️ **Edit (pencil icon)**

Make your changes, for example:
<h1>Hello Updated World 🎉</h1>

4.
5. Scroll down
6. Click **“Commit changes”**

🎉 Done!

👉 Now Netlify will **automatically update your website**

💻 Method 2: Using Git (Slightly Advanced)

If you are using your computer:

Step 1: Open terminal in your project folder

Step 2: Run these commands:

```
git add index.html
git commit -m "Updated index file"
git push
```

🔁 What Happens After Push?

- GitHub receives your updated code
- Netlify detects the change
- Netlify redeploys your website automatically 🚀

🧠 Simple Flow

Edit file → Commit → Push → Netlify auto deploy